

Then find the smallest element of $A[2 : n]$, and exchange it with $A[2]$. Then find the smallest element of $A[3 : n]$, and exchange it with $A[3]$. Continue in this manner for the first $n - 1$ elements of A . Write pseudocode for this algorithm, which is known as *selection sort*. What loop invariant does this algorithm maintain? Why does it need to run for only the first $n - 1$ elements, rather than for all n elements? Give the worst-case running time of selection sort in Θ -notation. Is the best-case running time any better?

2.2-3

Consider linear search again (see Exercise 2.1-4). How many elements of the input array need to be checked on the average, assuming that the element being searched for is equally likely to be any element in the array? How about in the worst case? Using Θ -notation, give the average-case and worst-case running times of linear search. Justify your answers.

2.2-4

How can you modify any sorting algorithm to have a good best-case running time?

2.3 Designing algorithms

You can choose from a wide range of algorithm design techniques. Insertion sort uses the *incremental* method: for each element $A[i]$, insert it into its proper place in the subarray $A[1 : i]$, having already sorted the subarray $A[1 : i - 1]$.

This section examines another design method, known as “divide-and-conquer,” which we explore in more detail in Chapter 4. We’ll use divide-and-conquer to design a sorting algorithm whose worst-case running time is much less than that of insertion sort. One advantage of using an algorithm that follows the divide-and-conquer method is that analyzing its running time is often straightforward, using techniques that we’ll explore in Chapter 4.

2.3.1 The divide-and-conquer method

Many useful algorithms are *recursive* in structure: to solve a given problem, they *recurse* (call themselves) one or more times to handle closely related subproblems. These algorithms typically follow the *divide-and-conquer* method: they break the problem into several subproblems that are similar to the original problem but smaller in size, solve the subproblems recursively, and then combine these solutions to create a solution to the original problem.

In the divide-and-conquer method, if the problem is small enough—the *base case*—you just solve it directly without recursing. Otherwise—the *recursive case*—you perform three characteristic steps:

Divide the problem into one or more subproblems that are smaller instances of the same problem.

Conquer the subproblems by solving them recursively.

Combine the subproblem solutions to form a solution to the original problem.

The *merge sort* algorithm closely follows the divide-and-conquer method. In each step, it sorts a subarray $A[p : r]$, starting with the entire array $A[1 : n]$ and recursing down to smaller and smaller subarrays. Here is how merge sort operates:

Divide the subarray $A[p : r]$ to be sorted into two adjacent subarrays, each of half the size. To do so, compute the midpoint q of $A[p : r]$ (taking the average of p and r), and divide $A[p : r]$ into subarrays $A[p : q]$ and $A[q + 1 : r]$.

Conquer by sorting each of the two subarrays $A[p : q]$ and $A[q + 1 : r]$ recursively using merge sort.

Combine by merging the two sorted subarrays $A[p : q]$ and $A[q + 1 : r]$ back into $A[p : r]$, producing the sorted answer.

The recursion “bottoms out”—it reaches the base case—when the subarray $A[p : r]$ to be sorted has just 1 element, that is, when p equals r . As we noted in the initialization argument for INSERTION-SORT’s loop invariant, a subarray comprising just a single element is always sorted.

The key operation of the merge sort algorithm occurs in the “combine” step, which merges two adjacent, sorted subarrays. The merge operation is performed by the auxiliary procedure `MERGE( $A, p, q, r$ )` on the following page, where A is an array and p, q , and r are indices into the array such that $p \leq q < r$. The procedure assumes that the adjacent subarrays $A[p : q]$ and $A[q + 1 : r]$ were already recursively sorted. It *merges* the two sorted subarrays to form a single sorted subarray that replaces the current subarray $A[p : r]$.

To understand how the `MERGE` procedure works, let’s return to our card-playing motif. Suppose that you have two piles of cards face up on a table. Each pile is sorted, with the smallest-value cards on top. You wish to merge the two piles into a single sorted output pile, which is to be face down on the table. The basic step consists of choosing the smaller of the two cards on top of the face-up piles, removing it from its pile—which exposes a new top card—and placing this card face down onto the output pile. Repeat this step until one input pile is empty, at which time you can just take the remaining input pile and flip over the entire pile, placing it face down onto the output pile.

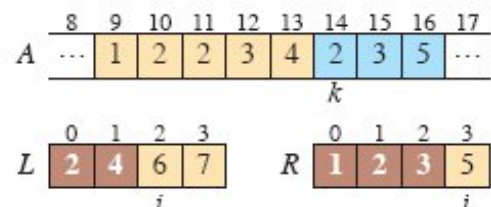
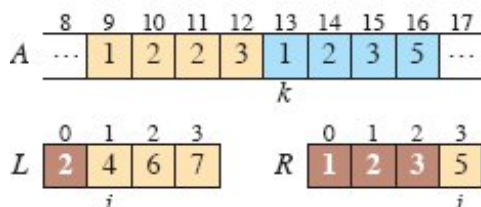
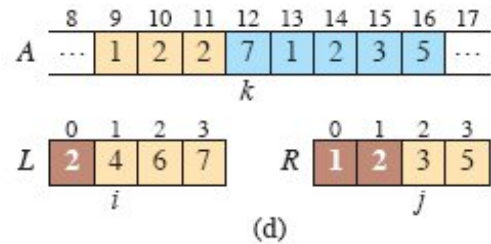
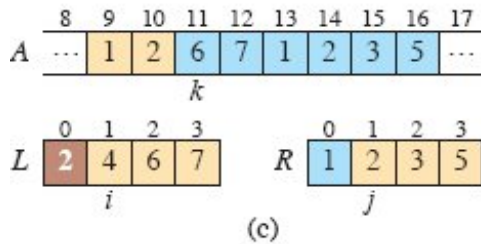
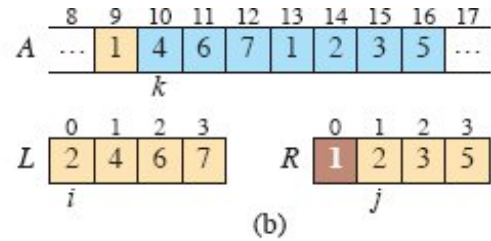
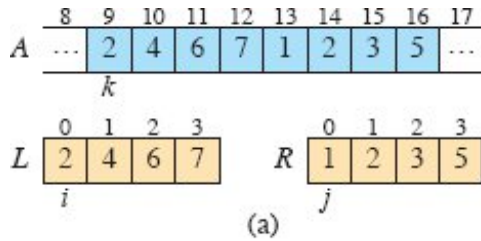
Let’s think about how long it takes to merge two sorted piles of cards. Each basic step takes constant time, since you are comparing just the two top cards. If the two sorted piles that you start with each have $n/2$ cards, then the number of basic steps is at least $n/2$ (since in whichever pile was emptied, every card was found to be smaller than some card from the other pile) and at most n (actually, at most $n - 1$, since after $n - 1$ basic steps, one of the piles must be empty). With each basic step taking constant time and the total number of basic steps being between $n/2$ and n , we can say that merging takes time roughly proportional to n . That is, merging takes $\Theta(n)$ time.

In detail, the `MERGE` procedure works as follows. It copies the two subarrays $A[p : q]$ and $A[q + 1 : r]$ into temporary arrays L and R (“left” and “right”), and then it merges the values in L and R back into $A[p : r]$. Lines 1 and 2 compute the lengths n_L and n_R of the subarrays $A[p : q]$ and $A[q + 1 : r]$, respectively. Then line 3 creates arrays $L[0 : n_L - 1]$ and $R[0 : n_R - 1]$ with respective lengths n_L and n_R .¹² The **for** loop of lines

4–5 copies the subarray $A[p : q]$ into L , and the **for** loop of lines 6–7 copies the subarray $A[q + 1 : r]$ into R .

```
MERGE( $A, p, q, r$ )
1  $n_L = q - p + 1$       // length of  $A[p : q]$ 
2  $n_R = r - q$           // length of  $A[q + 1 : r]$ 
3 let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4 for  $i = 0$  to  $n_L - 1$  // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5    $L[i] = A[p + i]$ 
6 for  $j = 0$  to  $n_R - 1$  // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7    $R[j] = A[q + j + 1]$ 
8  $i = 0$                 //  $i$  indexes the smallest remaining element in  $L$ 
9  $j = 0$                 //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged
    // element,
    // copy the smallest unmerged element back into  $A[p : r]$ .
12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18    $k = k + 1$ 
19 // Having gone through one of  $L$  and  $R$  entirely, copy the
    // remainder of the other to the end of  $A[p : r]$ .
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
```

Lines 8–18, illustrated in Figure 2.3, perform the basic steps. The **while** loop of lines 12–18 repeatedly identifies the smallest value in L and R that has yet to be copied back into $A[p : r]$ and copies it back in. As the comments indicate, the index k gives the position of A that is being filled in, and the indices i and j give the positions in L and R , respectively, of the smallest remaining values. Eventually, either all of L or all of R is copied back into $A[p : r]$, and this loop terminates. If the loop terminates because all of R has been copied back, that is, because j equals n_R , then i is still less than n_L , so that some of L has yet to be copied back, and these values are the greatest in both L and R . In this case, the **while** loop of lines 20–23 copies these remaining values of L into the last few positions of $A[p : r]$. Because j equals n_R , the **while** loop of lines 24–27 iterates 0 times. If instead the **while** loop of lines 12–18 terminates because i equals n_L , then all of L has already been copied back into $A[p : r]$, and the **while** loop of lines 24–27 copies the remaining values of R back into the end of $A[p : r]$.



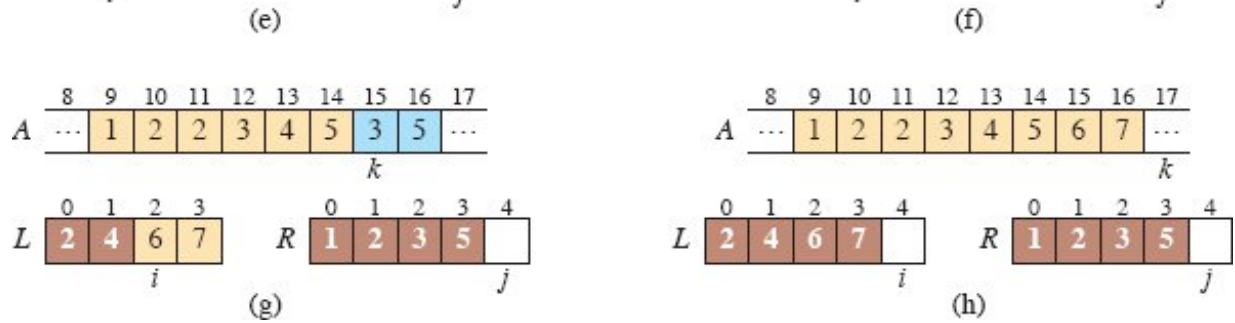


Figure 2.3 The operation of the **while** loop in lines 8–18 in the call `MERGE(A, 9, 12, 16)`, when the subarray $A[9 : 16]$ contains the values $\langle 2, 4, 6, 7, 1, 2, 3, 5 \rangle$. After allocating and copying into the arrays L and R , the array L contains $\langle 2, 4, 6, 7 \rangle$, and the array R contains $\langle 1, 2, 3, 5 \rangle$. Tan positions in A contain their final values, and tan positions in L and R contain values that have yet to be copied back into A . Taken together, the tan positions always comprise the values originally in $A[9 : 16]$. Blue positions in A contain values that will be copied over, and dark positions in L and R contain values that have already been copied back into A . (a)–(g) The arrays A , L , and R , and their respective indices k , i , and j prior to each iteration of the loop of lines 12–18. At the point in part (g), all values in R have been copied back into A (indicated by j equaling the length of R), and so the **while** loop in lines 12–18 terminates. (h) The arrays and indices at termination. The **while** loops of lines 20–23 and 24–27 copied back into A the remaining values in L and R , which are the largest values originally in $A[9 : 16]$. Here, lines 20–23 copied $L[2 : 3]$ into $A[15 : 16]$, and because all values in R had already been copied back into A , the **while** loop of lines 24–27 iterated 0 times. At this point, the subarray in $A[9 : 16]$ is sorted.

To see that the `MERGE` procedure runs in $\Theta(n)$ time, where $n = r - p + 1$,¹³ observe that each of lines 1–3 and 8–10 takes constant time, and the **for** loops of lines 4–7 take $\Theta(n_L + n_R) = \Theta(n)$ time.¹⁴ To account for the three **while** loops of lines 12–18, 20–23, and 24–27, observe that each iteration of these loops copies exactly one value from L or R back into A and that every value is copied back into A exactly once. Therefore, these three loops together make a total of n iterations. Since each iteration of each of the three loops takes constant time, the total time spent in these three loops is $\Theta(n)$.

We can now use the `MERGE` procedure as a subroutine in the merge sort algorithm. The procedure `MERGE-SORT(A, p, r)` on the facing page sorts the elements in the subarray $A[p : r]$. If p equals r , the subarray has just 1 element and is therefore already sorted. Otherwise, we must have $p < r$, and `MERGE-SORT` runs the divide, conquer, and combine steps. The divide step simply computes an index q that

partitions $A[p : r]$ into two adjacent subarrays: $A[p : q]$, containing $\lceil n/2 \rceil$ elements, and $A[q + 1 : r]$, containing $\lfloor n/2 \rfloor$ elements.¹⁵ The initial call $\text{MERGE-SORT}(A, 1, n)$ sorts the entire array $A[1 : n]$.

Figure 2.4 illustrates the operation of the procedure for $n = 8$, showing also the sequence of divide and merge steps. The algorithm recursively divides the array down to 1-element subarrays. The combine steps merge pairs of 1-element subarrays to form sorted subarrays of length 2, merges those to form sorted subarrays of length 4, and merges those to form the final sorted subarray of length 8. If n is not an exact power of 2, then some divide steps create subarrays whose lengths differ by 1. (For example, when dividing a subarray of length 7, one subarray has length 4 and the other has length 3.) Regardless of the lengths of the two subarrays being merged, the time to merge a total of n items is $\Theta(n)$.

```
MERGE-SORT( $A, p, r$ )
1  if  $p \geq r$                                 // zero or one element?
2      return
3   $q = \lfloor (p + r)/2 \rfloor$                       // midpoint of  $A[p : r]$ 
4  MERGE-SORT( $A, p, q$ )                        // recursively sort  $A[p : q]$ 
5  MERGE-SORT( $A, q + 1, r$ )                    // recursively sort  $A[q + 1 : r]$ 
6  // Merge  $A[p : q]$  and  $A[q + 1 : r]$  into  $A[p : r]$ .
7  MERGE( $A, p, q, r$ )
```

2.3.2 Analyzing divide-and-conquer algorithms

When an algorithm contains a recursive call, you can often describe its running time by a *recurrence equation* or *recurrence*, which describes the overall running time on a problem of size n in terms of the running time of the same algorithm on smaller inputs. You can then use mathematical tools to solve the recurrence and provide bounds on the performance of the algorithm.

A recurrence for the running time of a divide-and-conquer algorithm falls out from the three steps of the basic method. As we did for insertion sort, let $T(n)$ be the worst-case running time on a problem

of size n . If the problem size is small enough, say $n < n_0$ for some constant $n_0 > 0$, the straightforward solution takes constant time, which we write as $\Theta(1)$.¹⁶ Suppose that the division of the problem yields a subproblems, each with size n/b , that is, $1/b$ the size of the original. For merge sort, both a and b are 2, but we'll see other divide-and-conquer algorithms in which $a \neq b$. It takes $T(n/b)$ time to solve one subproblem of size n/b , and so it takes $aT(n/b)$ time to solve all a of them. If it takes $D(n)$ time to divide the problem into subproblems and $C(n)$ time to combine the solutions to the subproblems into the solution to the original problem, we get the recurrence

$$T(n) = \begin{cases} \Theta(1) & \text{if } n < n_0, \\ D(n) + aT(n/b) + C(n) & \text{otherwise.} \end{cases}$$

Chapter 4 shows how to solve common recurrences of this form.

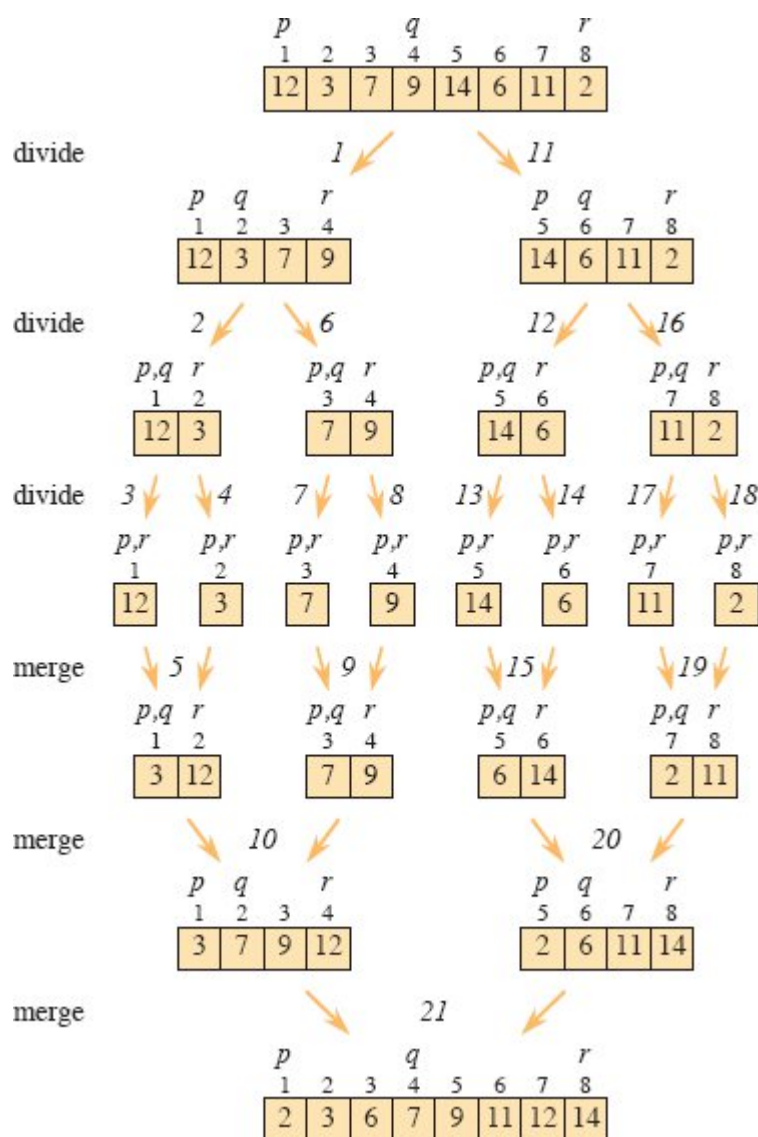


Figure 2.4 The operation of merge sort on the array A with length 8 that initially contains the sequence $\{12, 3, 7, 9, 14, 6, 11, 2\}$. The indices p , q , and r into each subarray appear above their values. Numbers in italics indicate the order in which the MERGE-SORT and MERGE procedures are called following the initial call of MERGE-SORT($A, 1, 8$).

Sometimes, the n/b size of the divide step isn't an integer. For example, the MERGE-SORT procedure divides a problem of size n into subproblems of sizes $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$. Since the difference between $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$ is at most 1, which for large n is much smaller than the effect of dividing n by 2, we'll squint a little and just call them both size $n/2$. As Chapter 4 will discuss, this simplification of ignoring floors and

ceilings does not generally affect the order of growth of a solution to a divide-and-conquer recurrence.

Another convention we'll adopt is to omit a statement of the base cases of the recurrence, which we'll also discuss in more detail in Chapter 4. The reason is that the base cases are pretty much always $T(n) = \Theta(1)$ if $n < n_0$ for some constant $n_0 > 0$. That's because the running time of an algorithm on an input of constant size is constant. We save ourselves a lot of extra writing by adopting this convention.

Analysis of merge sort

Here's how to set up the recurrence for $T(n)$, the worst-case running time of merge sort on n numbers.

Divide: The divide step just computes the middle of the subarray, which takes constant time. Thus, $D(n) = \Theta(1)$.

Conquer: Recursively solving two subproblems, each of size $n/2$, contributes $2T(n/2)$ to the running time (ignoring the floors and ceilings, as we discussed).

Combine: Since the MERGE procedure on an n -element subarray takes $\Theta(n)$ time, we have $C(n) = \Theta(n)$.

When we add the functions $D(n)$ and $C(n)$ for the merge sort analysis, we are adding a function that is $\Theta(n)$ and a function that is $\Theta(1)$. This sum is a linear function of n . That is, it is roughly proportional to n when n is large, and so merge sort's dividing and combining times together are $\Theta(n)$. Adding $\Theta(n)$ to the $2T(n/2)$ term from the conquer step gives the recurrence for the worst-case running time $T(n)$ of merge sort:

$$T(n) = 2T(n/2) + \Theta(n) . \quad (2.3)$$

Chapter 4 presents the “master theorem,” which shows that $T(n) = \Theta(n \lg n)$.¹⁷ Compared with insertion sort, whose worst-case running time is $\Theta(n^2)$, merge sort trades away a factor of n for a factor of $\lg n$. Because the logarithm function grows more slowly than any linear function, that's a good trade. For large enough inputs, merge sort, with its $\Theta(n \lg$

n) worst-case running time, outperforms insertion sort, whose worst-case running time is $\Theta(n^2)$.

We do not need the master theorem, however, to understand intuitively why the solution to recurrence (2.3) is $T(n) = \Theta(n \lg n)$. For simplicity, assume that n is an exact power of 2 and that the implicit base case is $n = 1$. Then recurrence (2.3) is essentially

$$T(n) = \begin{cases} c_1 & \text{if } n = 1, \\ 2T(n/2) + c_2n & \text{if } n > 1, \end{cases} \quad (2.4)$$

where the constant $c_1 > 0$ represents the time required to solve a problem of size 1, and $c_2 > 0$ is the time per array element of the divide and combine steps.¹⁸

Figure 2.5 illustrates one way of figuring out the solution to recurrence (2.4). Part (a) of the figure shows $T(n)$, which part (b) expands into an equivalent tree representing the recurrence. The c_2n term denotes the cost of dividing and combining at the top level of recursion, and the two subtrees of the root are the two smaller recurrences $T(n/2)$. Part (c) shows this process carried one step further by expanding $T(n/2)$. The cost for dividing and combining at each of the two nodes at the second level of recursion is $c_2n/2$. Continue to expand each node in the tree by breaking it into its constituent parts as determined by the recurrence, until the problem sizes get down to 1, each with a cost of c_1 . Part (d) shows the resulting *recursion tree*.

Next, add the costs across each level of the tree. The top level has total cost c_2n , the next level down has total cost $c_2(n/2) + c_2(n/2) = c_2n$, the level after that has total cost $c_2(n/4) + c_2(n/4) + c_2(n/4) + c_2(n/4) = c_2n$, and so on. Each level has twice as many nodes as the level above, but each node contributes only half the cost of a node from the level above. From one level to the next, doubling and halving cancel each other out, so that the cost across each level is the same: c_2n . In general, the level that is i levels below the top has 2^i nodes, each contributing a cost of $c_2(n/2^i)$, so that the i th level below the top has total cost $2^i \cdot$

$c_2(n/2^i) = c_2n$. The bottom level has n nodes, each contributing a cost of c_1 , for a total cost of c_1n .

The total number of levels of the recursion tree in Figure 2.5 is $\lg n + 1$, where n is the number of leaves, corresponding to the input size. An informal inductive argument justifies this claim. The base case occurs when $n = 1$, in which case the tree has only 1 level. Since $\lg 1 = 0$, we have that $\lg n + 1$ gives the correct number of levels. Now assume as an inductive hypothesis that the number of levels of a recursion tree with 2^i leaves is $\lg 2^i + 1 = i + 1$ (since for any value of i , we have that $\lg 2^i = i$). Because we assume that the input size is an exact power of 2, the next input size to consider is 2^{i+1} . A tree with $n = 2^{i+1}$ leaves has 1 more level than a tree with 2^i leaves, and so the total number of levels is $(i + 1) + 1 = \lg 2^{i+1} + 1$.

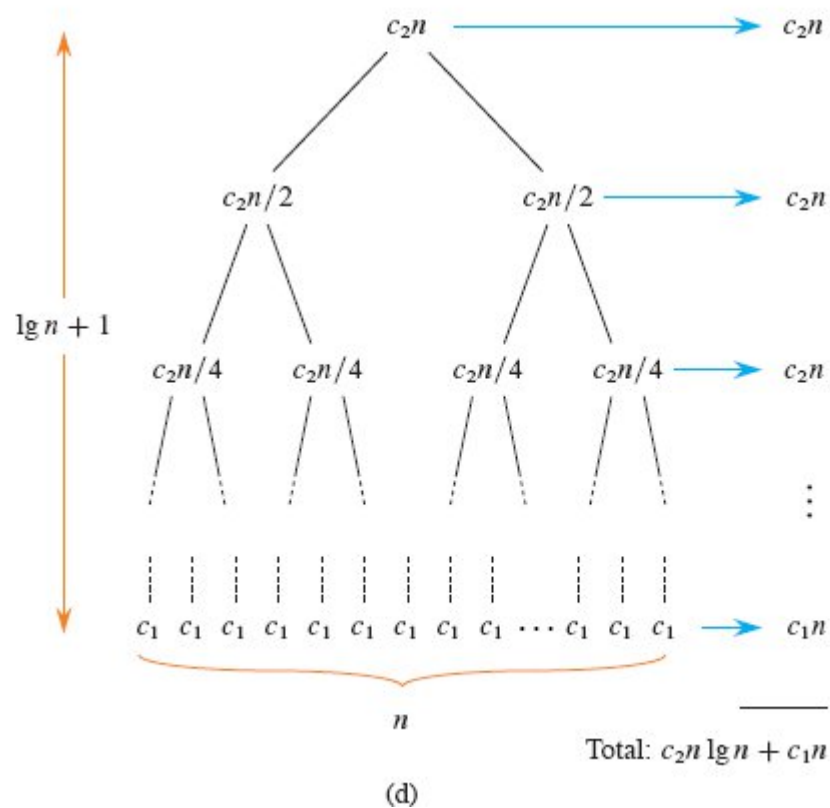
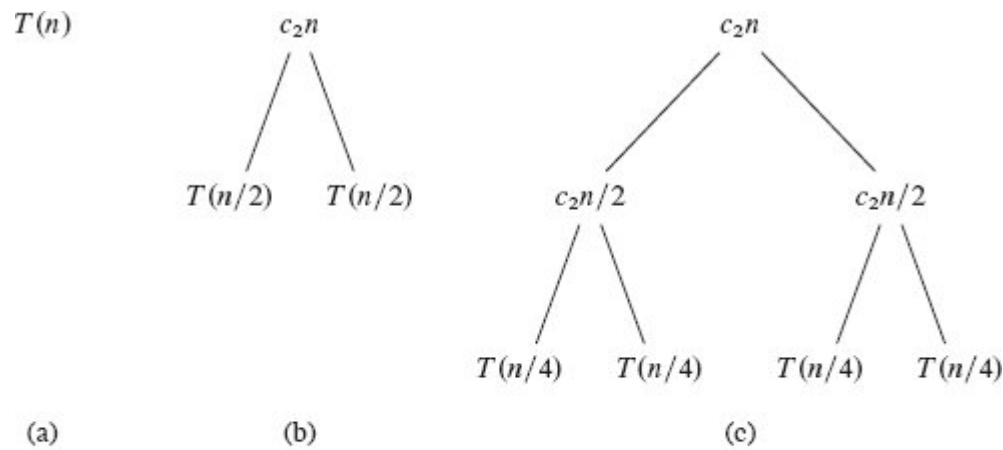


Figure 2.5 How to construct a recursion tree for the recurrence (2.4). Part (a) shows $T(n)$, which progressively expands in (b)–(d) to form the recursion tree. The fully expanded tree in part (d) has $\lg n + 1$ levels. Each level above the leaves contributes a total cost of c_2n , and the leaf level contributes c_1n . The total cost, therefore, is $c_2n \lg n + c_1n = \Theta(n \lg n)$.

To compute the total cost represented by the recurrence (2.4), simply add up the costs of all the levels. The recursion tree has $\lg n + 1$ levels. The levels above the leaves each cost c_2n , and the leaf level costs c_1n , for a total cost of $c_2n \lg n + c_1n = \Theta(n \lg n)$.

Exercises

2.3-1

Using Figure 2.4 as a model, illustrate the operation of merge sort on an array initially containing the sequence $\langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$.

2.3-2

The test in line 1 of the MERGE-SORT procedure reads “**if** $p \geq r$ ” rather than “**if** $p \neq r$.” If MERGE-SORT is called with $p > r$, then the subarray $A[p : r]$ is empty. Argue that as long as the initial call of MERGE-SORT($A, 1, n$) has $n \geq 1$, the test “**if** $p \neq r$ ” suffices to ensure that no recursive call has $p > r$.

2.3-3

State a loop invariant for the **while** loop of lines 12–18 of the MERGE procedure. Show how to use it, along with the **while** loops of lines 20–23 and 24–27, to prove that the MERGE procedure is correct.

2.3-4

Use mathematical induction to show that when $n \geq 2$ is an exact power of 2, the solution of the recurrence

$$T(n) = \begin{cases} 2 & \text{if } n = 2, \\ 2T(n/2) + n & \text{if } n > 2 \end{cases}$$

is $T(n) = n \lg n$.

2.3-5

You can also think of insertion sort as a recursive algorithm. In order to sort $A[1 : n]$, recursively sort the subarray $A[1 : n - 1]$ and then insert $A[n]$ into the sorted subarray $A[1 : n - 1]$. Write pseudocode for this

recursive version of insertion sort. Give a recurrence for its worst-case running time.

2.3-6

Referring back to the searching problem (see Exercise 2.1-4), observe that if the subarray being searched is already sorted, the searching algorithm can check the midpoint of the subarray against v and eliminate half of the subarray from further consideration. The *binary search* algorithm repeats this procedure, halving the size of the remaining portion of the subarray each time. Write pseudocode, either iterative or recursive, for binary search. Argue that the worst-case running time of binary search is $\Theta(\lg n)$.

2.3-7

The **while** loop of lines 5–7 of the INSERTION-SORT procedure in Section 2.1 uses a linear search to scan (backward) through the sorted subarray $A[1 : j - 1]$. What if insertion sort used a binary search (see Exercise 2.3-6) instead of a linear search? Would that improve the overall worst-case running time of insertion sort to $\Theta(n \lg n)$?

2.3-8

Describe an algorithm that, given a set S of n integers and another integer x , determines whether S contains two elements that sum to exactly x . Your algorithm should take $\Theta(n \lg n)$ time in the worst case.

Problems

2-1 *Insertion sort on small arrays in merge sort*

Although merge sort runs in $\Theta(n \lg n)$ worst-case time and insertion sort runs in $\Theta(n^2)$ worst-case time, the constant factors in insertion sort can make it faster in practice for small problem sizes on many machines. Thus it makes sense to *coarsen* the leaves of the recursion by using insertion sort within merge sort when subproblems become sufficiently small. Consider a modification to merge sort in which n/k sublists of